

```

//nweb01.c wmh 2011-04-20 adds .mp3 file type
//nweb.c nigel griffiths 2004-06-10 'A Tiny, Safe Webserver' (http://www.ibm.com/developerworks/systems/library/es-nweb/index.html)
//wmh 2010-04-19 comments:
// -compiles on glue with: gcc -Wall /usr/lib/libsocket.so /usr/lib/libnsl.so nweb.c -o nweb.exe
// (or per Beej) "" : gcc -Wall -o nweb.exe nweb.c -lsocket -lresolv -lnsl
// -runs on glue with (for example--port and directory are probably different): ./nweb.exe 8811 ~/150/_0419/nweb00
// -creates a server daemon which should be killed. Use '(glue)% ps -fu yourusername' to list your processes, then '(glue)% kill
<PIDofnweb>'
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <errno.h>
#include <string.h>
#include <fcntl.h>
#include <signal.h>
#include <sys/types.h>
#include <sys/socket.h>
#include <netinet/in.h>
#include <arpa/inet.h>

#define BUFSIZE 8096
#define ERROR 42
#define SORRY 43
#define LOG 44

struct {
    char *ext;
    char *filetype;
} extensions [] = {
    {"gif", "image/gif" },
    {"jpg", "image/jpeg"},
    {"jpeg", "image/jpeg"},
    {"png", "image/png" },
    {"zip", "image/zip" },
    {"gz", "image/gz" },
    {"tar", "image/tar" },
    {"htm", "text/html" },
    {"html", "text/html" },
    {"mp3", "music/mp3"},
    {0,0} };

void log(int type, char *s1, char *s2, int num)
{
    int fd ;
    char logbuffer[BUFSIZE*2];

    switch (type) {
    case ERROR: (void)sprintf(logbuffer, "ERROR: %s:%s Errno=%d exiting pid=%d", s1, s2, errno, getpid()); break;
    case SORRY:
        (void)sprintf(logbuffer, "<HTML><BODY><H1>nweb Web Server Sorry: %s %s</H1></BODY></HTML>\r\n", s1, s2);
        (void)write(num, logbuffer, strlen(logbuffer));
        (void)sprintf(logbuffer, "SORRY: %s:%s", s1, s2);
        break;
    case LOG: (void)sprintf(logbuffer, " INFO: %s:%s:%d", s1, s2, num); break;
    }
    /* no checks here, nothing can be done a failure anyway */
    if((fd = open("nweb.log", O_CREAT | O_WRONLY | O_APPEND, 0644)) >= 0) {
        (void)write(fd, logbuffer, strlen(logbuffer));
        (void)write(fd, "\n", 1);
        (void)close(fd);
    }
    if(type == ERROR || type == SORRY) exit(3);
}

/* this is a child web server process, so we can exit on errors */
void web(int fd, int hit)
{
    int j, file_fd, buflen, len;
    long i, ret;
    char * fstr;
    static char buffer[BUFSIZE+1]; /* static so zero filled */

    ret = read(fd, buffer, BUFSIZE); /* read Web request in one go */
    if(ret == 0 || ret == -1) { /* read failure stop now */
        log(SORRY, "failed to read browser request", "", fd);
    }

```

```

if(ret > 0 && ret < BUFSIZE) /* return code is valid chars */
    buffer[ret]=0; /* terminate the buffer */
else buffer[0]=0;

for(i=0;i<ret;i++) /* remove CF and LF characters */
    if(buffer[i] == '\r' || buffer[i] == '\n')
        buffer[i]='*';
log(LOG,"request",buffer,hit)

if( strcmp(buffer,"GET ",4) && strcmp(buffer,"get ",4) )
    log(SORRY,"Only simple GET operation supported",buffer,fd);

for(i=4;i<BUFSIZE;i++) { /* null terminate after the second space to ignore extra stuff */
    if(buffer[i] == ' ') { /* string is "GET URL " +lots of other stuff */
        buffer[i] = 0;
        break;
    }
}

for(j=0;j<i-1;j++) /* check for illegal parent directory use .. */
    if(buffer[j] == '.' && buffer[j+1] == '.')
        log(SORRY,"Parent directory (..) path names not supported",buffer,fd);

if( !strcmp(&buffer[0],"GET /\0",6) || !strcmp(&buffer[0],"get /\0",6) ) /* convert no filename to index file */
    (void)strcpy(buffer,"GET /index.html");

/* work out the file type and check we support it */
buflen=strlen(buffer);
fstr = (char *)0;
for(i=0;extensions[i].ext != 0;i++) {
    len = strlen(extensions[i].ext);
    if( !strcmp(&buffer[buflen-len], extensions[i].ext, len)) {
        fstr =extensions[i].filetype;
        break;
    }
}
if(fstr == 0) log(SORRY,"file extension type not supported",buffer,fd);

if(( file_fd = open(&buffer[5],O_RDONLY)) == -1) /* open the file for reading */
    log(SORRY, "failed to open file",&buffer[5],fd);

log(LOG,"SEND",&buffer[5],hit);

(void)sprintf(buffer,"HTTP/1.0 200 OK\r\nContent-Type: %s\r\n\r\n", fstr);
(void)write(fd,buffer,strlen(buffer));

/* send file in 8KB block - last block may be smaller */
while ( (ret = read(file_fd, buffer, BUFSIZE)) > 0 ) {
    (void)write(fd,buffer,ret);
}

#ifdef LINUX
    sleep(1); /* to allow socket to drain */
#endif
    exit(1);
}

int main(int argc, char **argv)
{
    int i, port, pid, listenfd, socketfd, hit;
    size_t length;
    static struct sockaddr_in cli_addr; /* static = initialised to zeros */
    static struct sockaddr_in serv_addr; /* static = initialised to zeros */

    if( argc < 3 || argc > 3 || !strcmp(argv[1], "-?") ) {
        (void)printf("hint: nweb Port-Number Top-Directory\n\n");
        "\tnweb is a small and very safe mini web server\n"
        "\tnweb only servers out file/web pages with extensions named below\n"
        "\tand only from the named directory or its sub-directories.\n"
        "\tThere is no fancy features = safe and secure.\n"
        "\tExample: nweb 8181 /home/nwebdir &\n"
        "\tOnly Supports:");
        for(i=0;extensions[i].ext != 0;i++)
            (void)printf(" %s",extensions[i].ext);

        (void)printf("\n\tNot Supported: URLs including \"..\", Java, Javascript, CGI\n");
    }
}

```

```

"\tNot Supported: directories / /etc /bin /lib /tmp /usr /dev /sbin \n"
"\tNo warranty given or implied\n\tNigel Griffiths nag@uk.ibm.com\n"
);
exit(0);
}
if( !strcmp(argv[2],"/" ,2 ) || !strcmp(argv[2],"/etc", 5 ) ||
    !strcmp(argv[2],"/bin",5 ) || !strcmp(argv[2],"/lib", 5 ) ||
    !strcmp(argv[2],"/tmp",5 ) || !strcmp(argv[2],"/usr", 5 ) ||
    !strcmp(argv[2],"/dev",5 ) || !strcmp(argv[2],"/sbin",6) ){
    (void)printf("ERROR: Bad top directory %s, see nweb -? \n",argv[2]);
    exit(3);
}
if(chdir(argv[2]) == -1){
    (void)printf("ERROR: Can't Change to directory %s\n",argv[2]);
    exit(4);
}

/* Become daemon + unstoppable and no zombies children (= no wait()) */
if(fork() != 0)
    return 0; /* parent returns OK to shell */
(void)signal(SIGCLD, SIG_IGN); /* ignore child death */
(void)signal(SIGHUP, SIG_IGN); /* ignore terminal hangups */
for(i=0;i<32;i++)
    (void)close(i); /* close open files */
(void)setpgpr(); /* break away from process group */

log(LOG,"nweb starting",argv[1],getpid());
/* setup the network socket */
if((listenfd = socket(AF_INET, SOCK_STREAM,0)) < 0)
    log(ERROR, "system call","socket",0);
port = atoi(argv[1]);
if(port < 0 || port > 60000)
    log(ERROR,"Invalid port number (try 1->60000)",argv[1],0);
serv_addr.sin_family = AF_INET;
serv_addr.sin_addr.s_addr = htonl(INADDR_ANY);
serv_addr.sin_port = htons(port);
if(bind(listenfd, (struct sockaddr *)&serv_addr,sizeof(serv_addr)) < 0)
    log(ERROR,"system call","bind",0);
if( listen(listenfd,64) < 0)
    log(ERROR,"system call","listen",0);

for(hit=1; ;hit++) {
    length = sizeof(cli_addr);
    if((socketfd = accept(listenfd, (struct sockaddr *)&cli_addr, &length)) < 0)
        log(ERROR,"system call","accept",0);

    if((pid = fork()) < 0) {
        log(ERROR,"system call","fork",0);
    }
    else {
        if(pid == 0) { /* child */
            (void)close(listenfd);
            web(socketfd,hit); /* never returns */
        } else { /* parent */
            (void)close(socketfd);
        }
    }
}
}
}

```